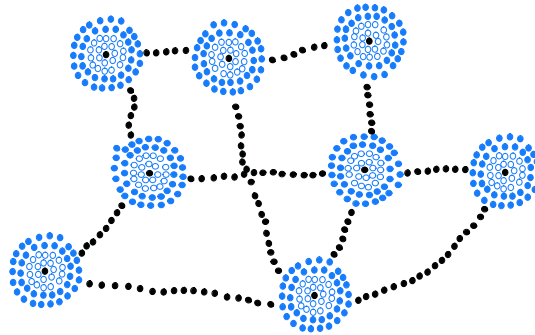




Contents

14.1 Graphs	612
14.1.1 The Graph ADT	618
14.2 Data Structures for Graphs	619
14.2.1 Edge List Structure	620
14.2.2 Adjacency List Structure	622
14.2.3 Adjacency Map Structure	624
14.2.4 Adjacency Matrix Structure	625
14.2.5 Java Implementation	626
14.3 Graph Traversals	630
14.3.1 Depth-First Search	631
14.3.2 DFS Implementation and Extensions	636
14.3.3 Breadth-First Search	640
14.4 Transitive Closure	643
14.5 Directed Acyclic Graphs	647
14.5.1 Topological Ordering	647
14.6 Shortest Paths	651
14.6.1 Weighted Graphs	651
14.6.2 Dijkstra's Algorithm	653
14.7 Minimum Spanning Trees	662
14.7.1 Prim-Jarník Algorithm	664
14.7.2 Kruskal's Algorithm	667
14.7.3 Disjoint Partitions and Union-Find Structures	672
14.8 Exercises	677

14.1 Graphs

A **graph** is a way of representing relationships that exist between pairs of objects. That is, a graph is a set of objects, called vertices, together with a collection of pairwise connections between them, called edges. Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering. By the way, this notion of a “graph” should not be confused with bar charts and function plots, as these kinds of “graphs” are unrelated to the topic of this chapter.

Viewed abstractly, a **graph** G is simply a set V of **vertices** and a collection E of pairs of vertices from V , called **edges**. Thus, a graph is a way of representing connections or relationships between pairs of objects from some set V . Incidentally, some books use different terminology for graphs and refer to what we call vertices as **nodes** and what we call edges as **arcs**. We use the terms “vertices” and “edges.”

Edges in a graph are either **directed** or **undirected**. An edge (u, v) is said to be **directed** from u to v if the pair (u, v) is ordered, with u preceding v . An edge (u, v) is said to be **undirected** if the pair (u, v) is not ordered. Undirected edges are sometimes denoted with set notation, as $\{u, v\}$, but for simplicity we use the pair notation (u, v) , noting that in the undirected case (u, v) is the same as (v, u) . Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles. The following are some examples of directed and undirected graphs.

Example 14.1: We can visualize collaborations among the researchers of a certain discipline by constructing a graph whose vertices are associated with the researchers themselves, and whose edges connect pairs of vertices associated with researchers who have coauthored a paper or book. (See Figure 14.1.) Such edges are undirected because coauthorship is a **symmetric** relation; that is, if A has coauthored something with B , then B necessarily has coauthored something with A .

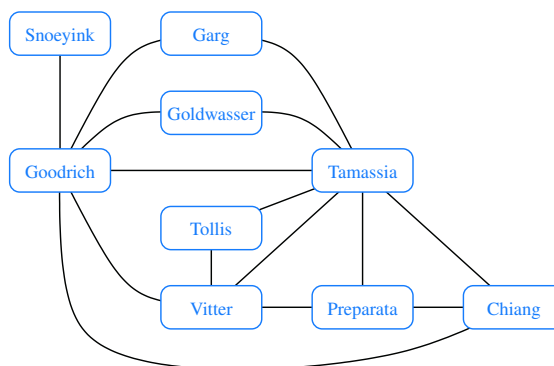


Figure 14.1: Graph of coauthorship among some authors.

Example 14.2: We can associate with an object-oriented program a graph whose vertices represent the classes defined in the program, and whose edges indicate inheritance between classes. There is an edge from a vertex v to a vertex u if the class for v inherits from the class for u . Such edges are directed because the inheritance relation only goes in one direction (that is, it is **asymmetric**).

If all the edges in a graph are undirected, then we say the graph is an **undirected graph**. Likewise, a **directed graph**, also called a **digraph**, is a graph whose edges are all directed. A graph that has both directed and undirected edges is often called a **mixed graph**. Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u, v) by the pair of directed edges (u, v) and (v, u) . It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications, as in the following example.

Example 14.3: A city map can be modeled as a graph whose vertices are intersections or dead ends, and whose edges are stretches of streets without intersections. This graph has both undirected edges, which correspond to stretches of two-way streets, and directed edges, which correspond to stretches of one-way streets. Thus, in this way, a graph modeling a city map is a mixed graph.

Example 14.4: Physical examples of graphs are present in the electrical wiring and plumbing networks of a building. Such networks can be modeled as graphs, where each connector, fixture, or outlet is viewed as a vertex, and each uninterrupted stretch of wire or pipe is viewed as an edge. Such graphs are actually components of much larger graphs, namely the local power and water distribution networks. Depending on the specific aspects of these graphs that we are interested in, we may consider their edges as undirected or directed, for, in principle, water can flow in a pipe and current can flow in a wire in either direction.

The two vertices joined by an edge are called the **end vertices** (or **endpoints**) of the edge. If an edge is directed, its first endpoint is its **origin** and the other is the **destination** of the edge. Two vertices u and v are said to be **adjacent** if there is an edge whose end vertices are u and v . An edge is said to be **incident** to a vertex if the vertex is one of the edge's endpoints. The **outgoing edges** of a vertex are the directed edges whose origin is that vertex. The **incoming edges** of a vertex are the directed edges whose destination is that vertex. The **degree** of a vertex v , denoted $\deg(v)$, is the number of incident edges of v . The **in-degree** and **out-degree** of a vertex v are the number of the incoming and outgoing edges of v , and are denoted $\text{indeg}(v)$ and $\text{outdeg}(v)$, respectively.

Example 14.5: We can study air transportation by constructing a graph G , called a **flight network**, whose vertices are associated with airports, and whose edges are associated with flights. (See Figure 14.2.) In graph G , the edges are directed because a given flight has a specific travel direction. The endpoints of an edge e in G correspond respectively to the origin and destination of the flight corresponding to e . Two airports are adjacent in G if there is a flight that flies between them, and an edge e is incident to a vertex v in G if the flight for e flies to or from the airport for v . The outgoing edges of a vertex v correspond to the outbound flights from v 's airport, and the incoming edges correspond to the inbound flights to v 's airport. Finally, the in-degree of a vertex v of G corresponds to the number of inbound flights to v 's airport, and the out-degree of a vertex v in G corresponds to the number of outbound flights.

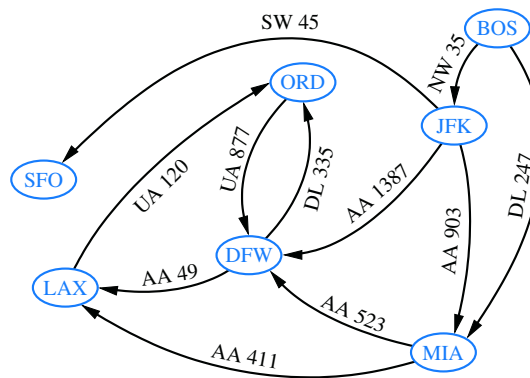


Figure 14.2: Example of a directed graph representing a flight network. The endpoints of edge UA 120 are LAX and ORD; hence, LAX and ORD are adjacent. The in-degree of DFW is 3, and the out-degree of DFW is 2.

The definition of a graph refers to the group of edges as a **collection**, not a **set**, thus allowing two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called **parallel edges** or **multiple edges**. A flight network can contain parallel edges (Example 14.5), such that multiple edges between the same pair of vertices could indicate different flights operating on the same route at different times of the day. Another special type of edge is one that connects a vertex to itself. Namely, we say that an edge (undirected or directed) is a **self-loop** if its two endpoints coincide. A self-loop may occur in a graph associated with a city map (Example 14.3), where it would correspond to a “circle” (a curving street that returns to its starting point).

With few exceptions, graphs do not have parallel edges or self-loops. Such graphs are said to be **simple**. Thus, we can usually say that the edges of a simple graph are a **set** of vertex pairs (and not just a collection). Throughout this chapter, we will assume that a graph is simple unless otherwise specified.

A **path** is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex such that each edge is incident to its predecessor and successor vertex. A **cycle** is a path that starts and ends at the same vertex, and that includes at least one edge. We say that a path is **simple** if each vertex in the path is distinct, and we say that a cycle is **simple** if each vertex in the cycle is distinct, except for the first and last one. A **directed path** is a path such that all edges are directed and are traversed along their direction. A **directed cycle** is similarly defined. For example, in Figure 14.2, (BOS, NW 35, JFK, AA 1387, DFW) is a directed simple path, and (LAX, UA 120, ORD, UA 877, DFW, AA 49, LAX) is a directed simple cycle. Note that a directed graph may have a cycle consisting of two edges with opposite direction between the same pair of vertices, for example (ORD, UA 877, DFW, DL 335, ORD) in Figure 14.2. A directed graph is **acyclic** if it has no directed cycles. For example, if we were to remove the edge UA 877 from the graph in Figure 14.2, the remaining graph is acyclic. If a graph is simple, we may omit the edges when describing path P or cycle C , as these are well defined, in which case P is a list of adjacent vertices and C is a cycle of adjacent vertices.

Example 14.6: Given a graph G representing a city map (see Example 14.3), we can model a couple driving to dinner at a recommended restaurant as traversing a path through G . If they know the way, and do not accidentally go through the same intersection twice, then they traverse a simple path in G . Likewise, we can model the entire trip the couple takes, from their home to the restaurant and back, as a cycle. If they go home from the restaurant in a completely different way than how they went, not even going through the same intersection twice, then their entire round trip is a simple cycle. Finally, if they travel along one-way streets for their entire trip, we can model their night out as a directed cycle.

Given vertices u and v of a (directed) graph G , we say that u **reaches** v , and that v is **reachable** from u , if G has a (directed) path from u to v . In an undirected graph, the notion of **reachability** is symmetric, that is to say, u reaches v if and only if v reaches u . However, in a directed graph, it is possible that u reaches v but v does not reach u , because a directed path must be traversed according to the respective directions of the edges. A graph is **connected** if, for any two vertices, there is a path between them. A directed graph $\vec{G}$ is **strongly connected** if for any two vertices u and v of $\vec{G}$, u reaches v and v reaches u . (See Figure 14.3 for some examples.)

A **subgraph** of a graph G is a graph H whose vertices and edges are subsets of the vertices and edges of G , respectively. A **spanning subgraph** of G is a subgraph of G that contains all the vertices of the graph G . If a graph G is not connected, its maximal connected subgraphs are called the **connected components** of G . A **forest** is a graph without cycles. A **tree** is a connected forest, that is, a connected graph without cycles. A **spanning tree** of a graph is a spanning subgraph that is a tree. (Note that this definition of a tree is somewhat different from the one given in Chapter 8, as there is not necessarily a designated root.)

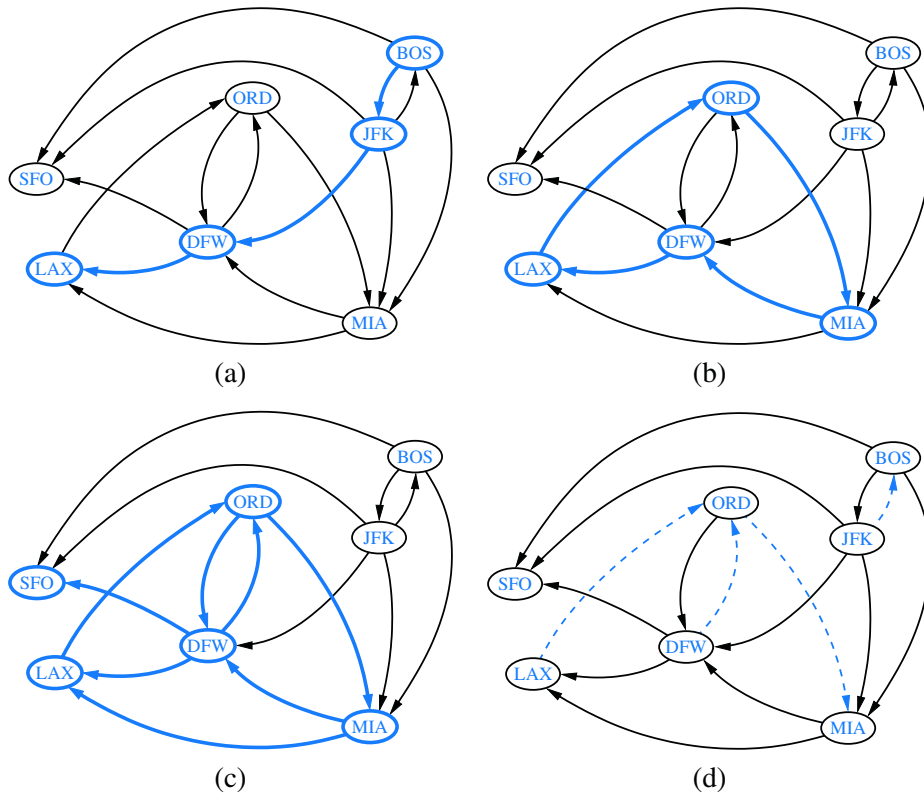


Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in a directed acyclic graph.

Example 14.7: Perhaps the most talked about graph today is the Internet, which can be viewed as a graph whose vertices are computers and whose (undirected) edges are communication connections between pairs of computers on the Internet. The computers and the connections between them in a single domain, like wiley.com, form a subgraph of the Internet. If this subgraph is connected, then two users on computers in this domain can send email to one another without having their information packets ever leave their domain. Suppose the edges of this subgraph form a spanning tree. This implies that, if even a single connection goes down (for example, because someone pulls a communication cable out of the back of a computer in this domain), then this subgraph will no longer be connected.

In the propositions that follow, we explore a few important properties of graphs.

Proposition 14.8: *If G is a graph with m edges and vertex set V , then*

$$\sum_{v \in V} \deg(v) = 2m.$$

Justification: An edge (u, v) is counted twice in the summation above; once by its endpoint u and once by its endpoint v . Thus, the total contribution of the edges to the degrees of the vertices is twice the number of edges. ■

Proposition 14.9: *If G is a directed graph with m edges and vertex set V , then*

$$\sum_{v \in V} \text{indeg}(v) = \sum_{v \in V} \text{outdeg}(v) = m.$$

Justification: In a directed graph, an edge (u, v) contributes one unit to the out-degree of its origin u and one unit to the in-degree of its destination v . Thus, the total contribution of the edges to the out-degrees of the vertices is equal to the number of edges, and similarly for the in-degrees. ■

We next show that a simple graph with n vertices has $O(n^2)$ edges.

Proposition 14.10: *Let G be a simple graph with n vertices and m edges. If G is undirected, then $m \leq n(n-1)/2$, and if G is directed, then $m \leq n(n-1)$.*

Justification: Suppose that G is undirected. Since no two edges can have the same endpoints and there are no self-loops, the maximum degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.8, $2m \leq n(n-1)$. Now suppose that G is directed. Since no two edges can have the same origin and destination, and there are no self-loops, the maximum in-degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.9, $m \leq n(n-1)$. ■

There are a number of simple properties of trees, forests, and connected graphs.

Proposition 14.11: *Let G be an undirected graph with n vertices and m edges.*

- *If G is connected, then $m \geq n-1$.*
- *If G is a tree, then $m = n-1$.*
- *If G is a forest, then $m \leq n-1$.*

14.1.1 The Graph ADT

A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph. A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume the element can be retrieved with the `getElement()` method. An Edge also stores an associated object (e.g., a flight number, travel distance, cost), which is returned by its `getElement()` method.

The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either *undirected* or *directed*, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge $\{u, v\}$ as a pair of directed edges (u, v) and (v, u) . The Graph ADT includes the following methods:

- `numVertices()`: Returns the number of vertices of the graph.
- `vertices()`: Returns an iteration of all the vertices of the graph.
- `numEdges()`: Returns the number of edges of the graph.
- `edges()`: Returns an iteration of all the edges of the graph.
- `getEdge( $u, v$ )`: Returns the edge from vertex u to vertex v , if one exists; otherwise return null. For an undirected graph, there is no difference between `getEdge( $u, v$ )` and `getEdge( $v, u$ )`.
- `endVertices( $e$ )`: Returns an array containing the two endpoint vertices of edge e . If the graph is directed, the first vertex is the origin and the second is the destination.
- `opposite( $v, e$ )`: For edge e incident to vertex v , returns the other vertex of the edge; an error occurs if e is not incident to v .
- `outDegree( $v$ )`: Returns the number of outgoing edges from vertex v .
- `inDegree( $v$ )`: Returns the number of incoming edges to vertex v . For an undirected graph, this returns the same value as does `outDegree( $v$ )`.
- `outgoingEdges( $v$ )`: Returns an iteration of all outgoing edges from vertex v .
- `incomingEdges( $v$ )`: Returns an iteration of all incoming edges to vertex v . For an undirected graph, this returns the same collection as does `outgoingEdges( $v$ )`.
- `insertVertex( $x$ )`: Creates and returns a new Vertex storing element x .
- `insertEdge( $u, v, x$ )`: Creates and returns a new Edge from vertex u to vertex v , storing element x ; an error occurs if there already exists an edge from u to v .
- `removeVertex( $v$ )`: Removes vertex v and all its incident edges from the graph.
- `removeEdge( $e$ )`: Removes edge e from the graph.